

Eightfold Candidate Transformer Design

Pipeline Overview

1. Ingest

- Load each configured source file and parse it into a partial canonical record.
- Supported sources: recruiter CSV, ATS JSON, recruiter notes TXT.
- Each parser returns values with normalized forms, confidence, and provenance.

2. Normalize

- Standardize fields during parsing, not post-hoc.
- Email extraction normalizes to lowercase and validates.
- Phone parsing uses ``phonenumbers`` and outputs E.164.
- Dates normalize to ``YYYY-MM`` or ``YYYY`` when only year is present.
- Countries normalize to ISO-3166 alpha-2.
- Skills canonicalize to a known set and preserve normalized names.

3. Merge

- Combine partial records from different sources into a single canonical record.
- Scalar fields use the highest-confidence value.
- List fields deduplicate by normalized values, merging confidence and sources.
- Provenance of each extracted value is preserved in a ``provenance`` array.

4. Confidence

- Each field and list item stores a confidence score.
- The engine computes ``overall_confidence`` as the mean of all available confidences.
- Missing or invalid input does not crash; it instead emits null or omits values.

5. Projection

- The runtime config describes the desired output shape, mapping from canonical paths.
- Supports selecting fields, remapping with ``from``, per-field normalization, and type enforcement.
- Supports runtime toggles for ``include_confidence``, ``include_provenance``, and ``on_missing`` behavior.

6. Validation

- Output projection enforces field types and handles missing values according to config.
- If ``on_missing`` is ``error``, the pipeline raises a validation error.
- Default output always returns the canonical schema shape.

Canonical Output Schema

- ``candidate_id``: string
- ``full_name``: string
- ``emails``: string[]
- ``phones``: string[] (E.164)
- ``location``: { city, region, country }

- ``links``: { linkedin, github, portfolio, other[] }
- ``headline``: string | null
- ``years_experience``: number | null
- ``skills``: [{ name, confidence, sources[] }]
- ``experience``: [{ company, title, start, end, summary, confidence }]
- ``education``: [{ institution, degree, field, end_year }]
- ``provenance``: [{ field, source, method, value }]
- ``overall_confidence``: number

Merge & Confidence Policy

- Scalar fields win by confidence. If two sources disagree, the higher confidence value is kept.
- List elements are unified by normalized value; duplicates are merged and confidences kept at the maximum.
- We use reliable confidence heuristics: ATS JSON and notes have higher confidence for emails and skills than free-text resumes.
- Provenance is additive: every source contributing a field is recorded.
- If no candidate ID is present, one is generated deterministically from email and name.

Runtime Config Handling

- The engine reads ``fields`` from runtime config and projects each requested field.
- The ``from`` path resolves against the canonical record.
- Normalization is applied as requested; for example ``E164`` for phone or ``canonical`` for skill names.
- Type coercion is performed after normalization.
- ``include_confidence`` removes confidence metadata from the projection tree.
- ``include_provenance`` controls whether provenance is kept.
- ``on_missing`` controls missing-field behavior: ``null``, ``omit``, or ``error``.

Edge Cases

1. Missing source files
 - The parser returns a source error marker and the pipeline continues.
2. Malformed files
 - Parsing exceptions become source errors instead of crashing the whole pipeline.
3. Multiple conflicting emails
 - The engine keeps all unique normalized emails and uses confidence to select the canonical primary if projected from ``email``.
4. Free-text phone formats
 - The notes parser extracts phone patterns and normalizes them through the same E.164 pipeline.
5. Unknown skill names
 - Unknown skill tokens are normalized to title case rather than inventing canonical skill mappings.

Deliberate Scope Choices

- I support at least one structured source (`recruiter.csv`) and one unstructured source (`notes.txt`).
- I did not implement REST API scraping for GitHub or LinkedIn due to time and sample input constraints.
- I did not build a UI; the solution uses a thin CLI with file-based input and output.
- I treat experience and education as pass-through list records, with a focus on normalization and merge behavior.